

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	<i>Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.</i>		
Student Name:	Jeeva Raghul .M	Reg. No.:	192524487
Section / Batch:	B Tech AIDS	Date:	28/08/2026

Code of Conduct

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Linear probing vs separate chaining

Factor	Linear probing	Separate
clustering	causes primary clustering because positions	Doesn't cause primary because collisions
High load	Performance decreases as table will be full	Handles high load factors shared
Avg Search	$O(1)$ load factor is slow	$O(1)$ with good as functions
worst case	$O(n)$	$O(1)$

Conclusion:

The linear probing is memory efficient & cache friendly with separate chaining is more flexible & suitable for frequent changing datasets.

2).

Factor	Quick Sort	Merge Sort
Avg time	$O(n \log n)$	$O(n \log n)$
worst time	$O(n^2)$	$O(n \log n)$
Extra time	$O(\log n)$ avg recursion	$O(\log n)$ recursion merging can be done
stability	usually not suitable	stable

Quick Sort: It divides list around a pivot its avg complexity of collisions then quadratic.

1) clustering:

Quadratic probing reduces primary clustering, but keys with the same initial # values follow the same frequency.

Conclusion:

Double hashing is generally more efficient when the hash table has a high load factor.

Factor	Quadratic Probing	Double hashing
clustering	Avoid primary clustering	greatly reduces clustering
collision	uses quadratic jump to find an empty position	uses 2nd hash function to determine.
Dim tables	Performance decreases as load factor $\uparrow$	Performs better high load factor

Impact on Algorithm:-

- * Input size
- * Expected data arrangements
- * requirement
- * Available memory
- * Need for predictable performance

4) Best vs worst case:-

a) Practical Significance

Best - case complexity represents the minimum time an algorithm requires for favorable input... The worst case - time complexity represents the max time taken for most unfavorable input arrangements.

Algorithm	Best Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$
Selection	$O(n^2)$	$O(n^2)$
Insertion	$O(n)$	$O(n^2)$
Merge ✓	$O(n \log n)$	$O(n \log n)$
Quick	$O(n \log n)$	$O(n^2)$
Heap	$O(n \log n)$	$O(n \log n)$

Recursive vs Iterative Quick Sort

Factor	Recursive Sort	Iterative Sort
stack usage	uses function call stack	uses an explicit stack.
Avg Space	$O(\log n)$	$O(\log n)$ pop
worst case	$O(n)$	can be constructed using explicit stack
Implementation	Simple & Easy to understand.	More complex
Stack overflow	Higher risk for unbalanced position	lower its because recursion

a) space complexity:

Recursive Quick Sort requires stack space is recursive function calls balanced position give approx $O(\log n)$ stack usage.

b) Easy of implementation:-

Requires quick sort a easier because
the divide & conquer structure naturally fits
recursion alternative quick sort.

copy